

TOPIC: Elevator system

Introduction:

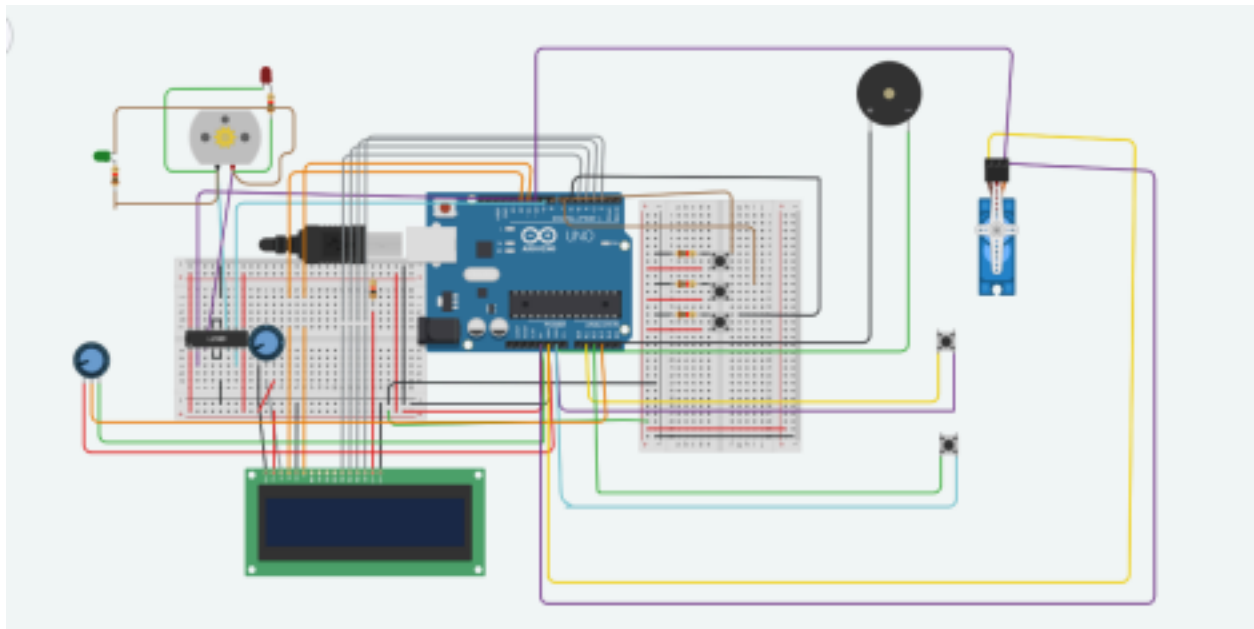
This project presents a working model of a three-floor elevator system developed using Arduino Uno on Tinkercad. The system simulates real-world elevator functions using digital components like push buttons for floor selection, LCD display, 7-segment display, servo motor for doors, and DC motors for movement. It also incorporates a potentiometer-based weight detection system and emergency controls to add realism and safety. The project aims to demonstrate core automation logic, floor handling, and safety mechanisms commonly used in modern elevator systems.

Components Used:

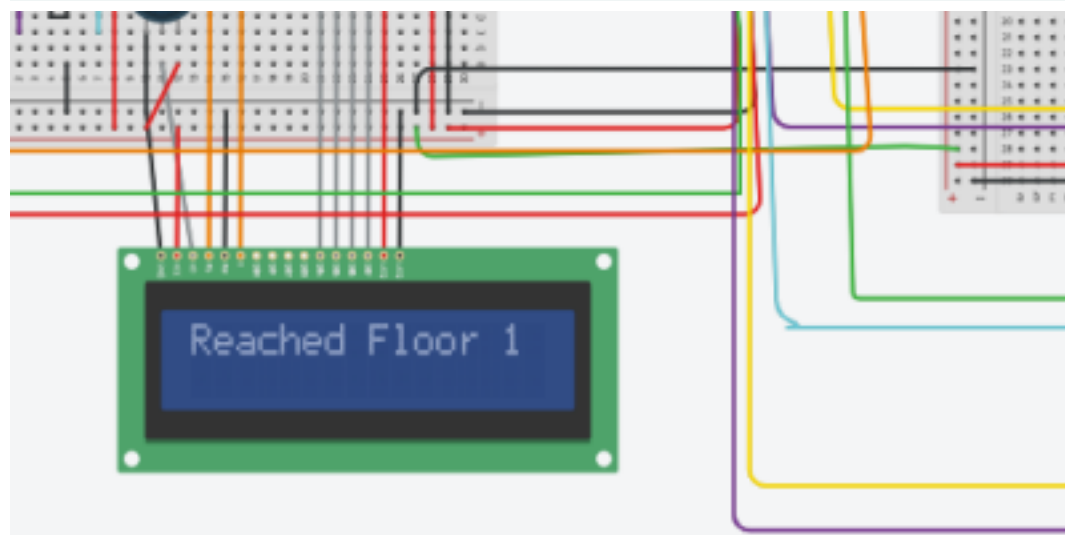
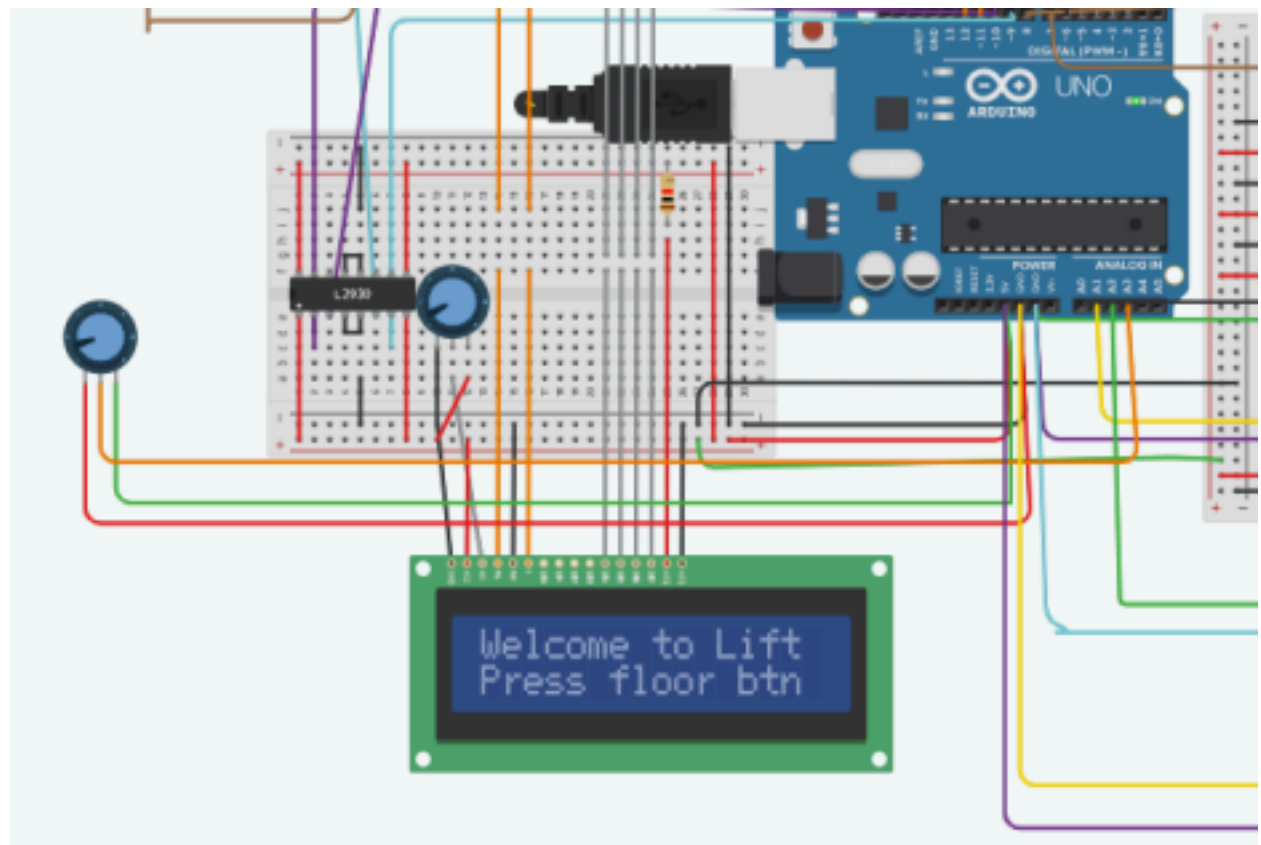
- *Arduino Uno R3 – 1*
- *Breadboard – 2*
- *16x2 LCD Display (with parallel interface) – 1*
- *Push Buttons – 5*
- *3 for floor selection*
- *2 emergency/fire button*
- *Potentiometers – 2*
- *DC Motor – 1*
- *L293D Motor Driver IC – 1*
- *Servo Motor (micro servo) – 1*
- *LEDs – 2*
- *1 Red led*
- *1 Green led*

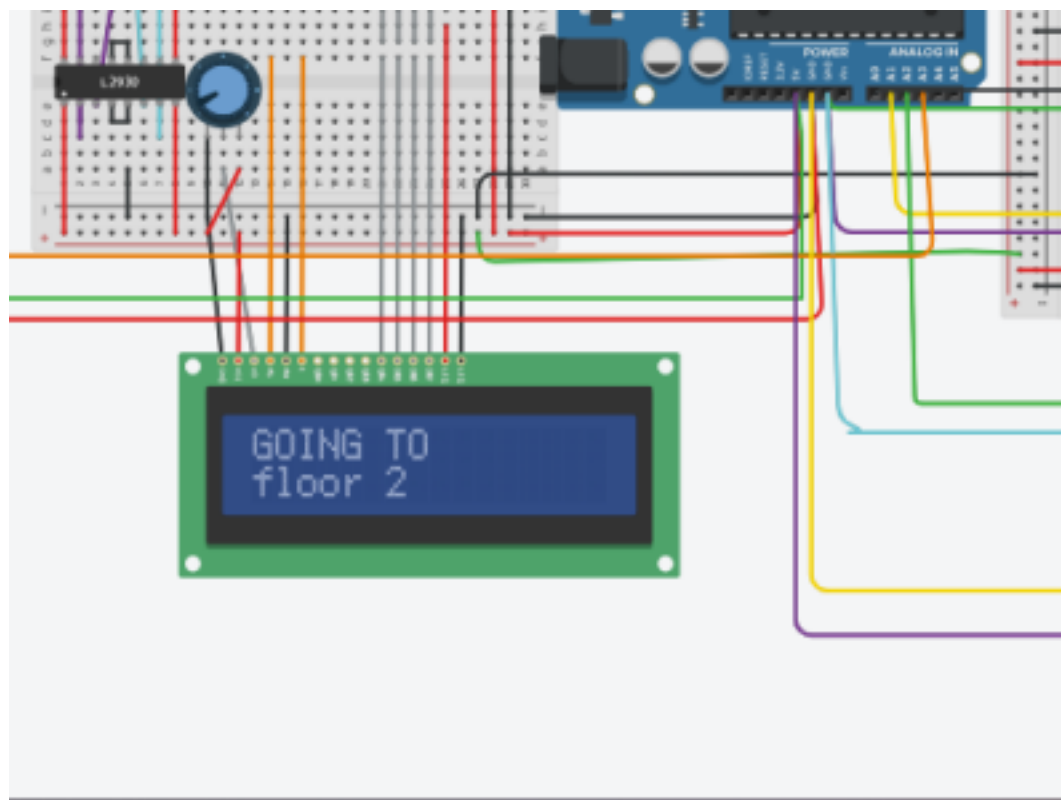
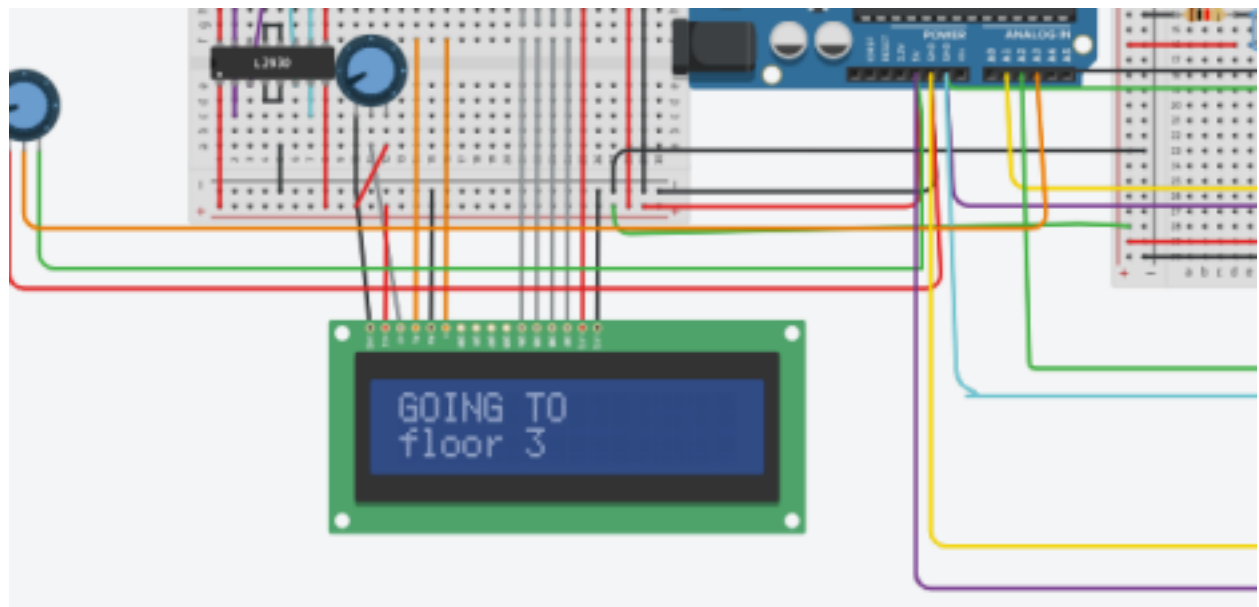
- *Piezo Buzzer – 1*
- *Resistors – Several (for LEDs and pull-down resistors for buttons)*
- *Jumper Wires – Multiple*
- *Power Supply – 1*

IMAGES

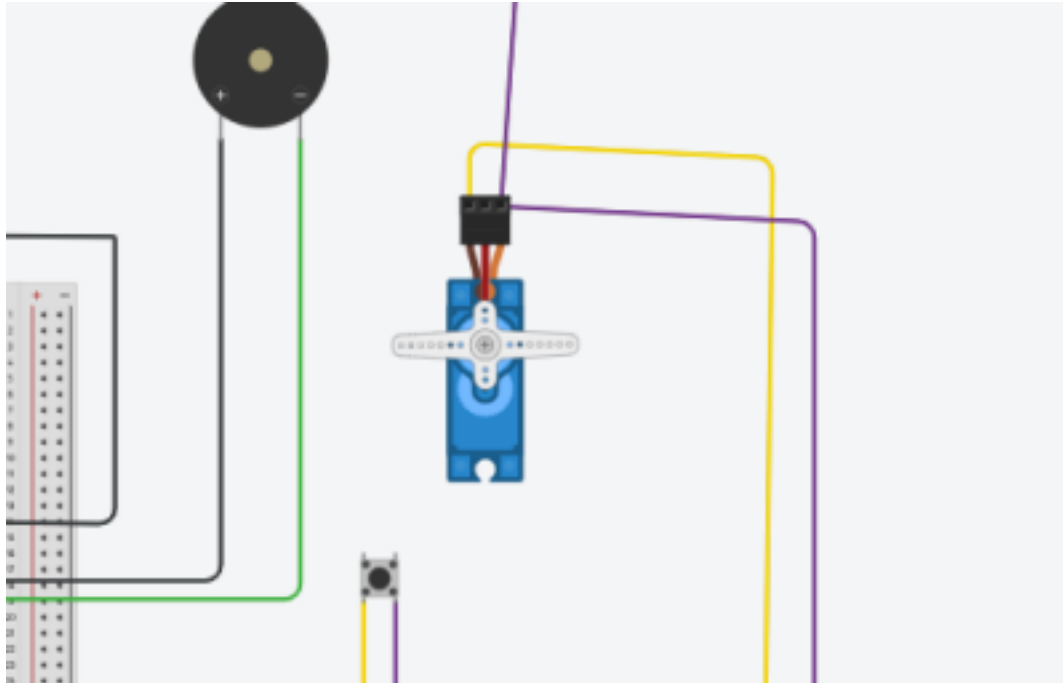


System Initialization on Simulation Start

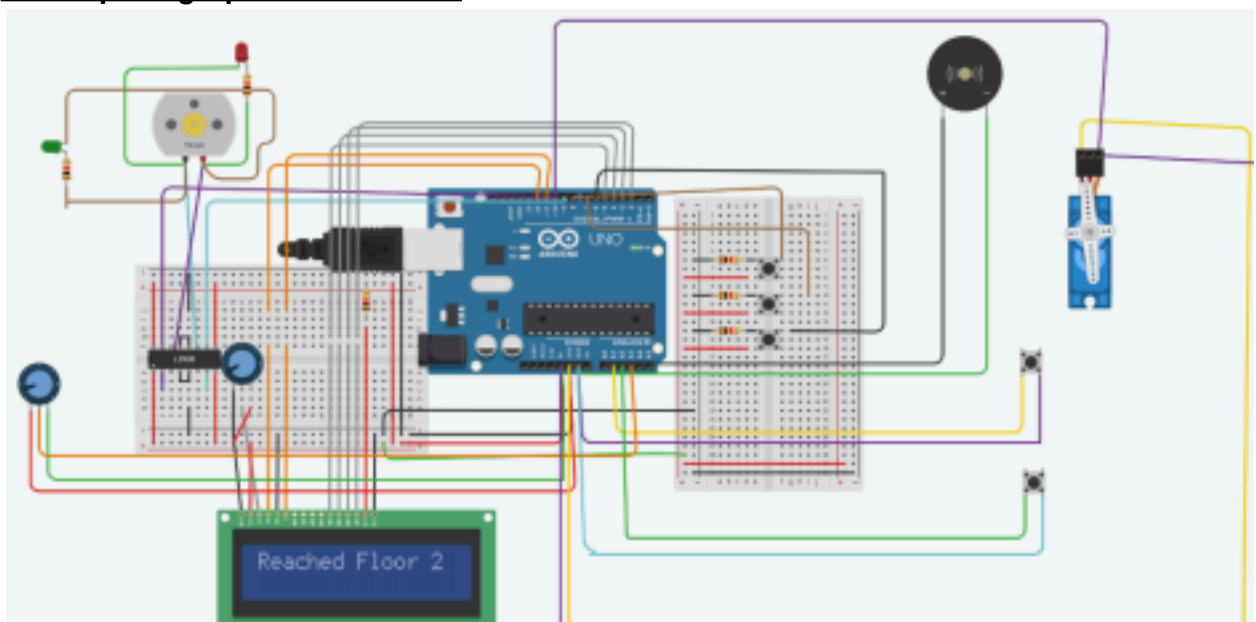




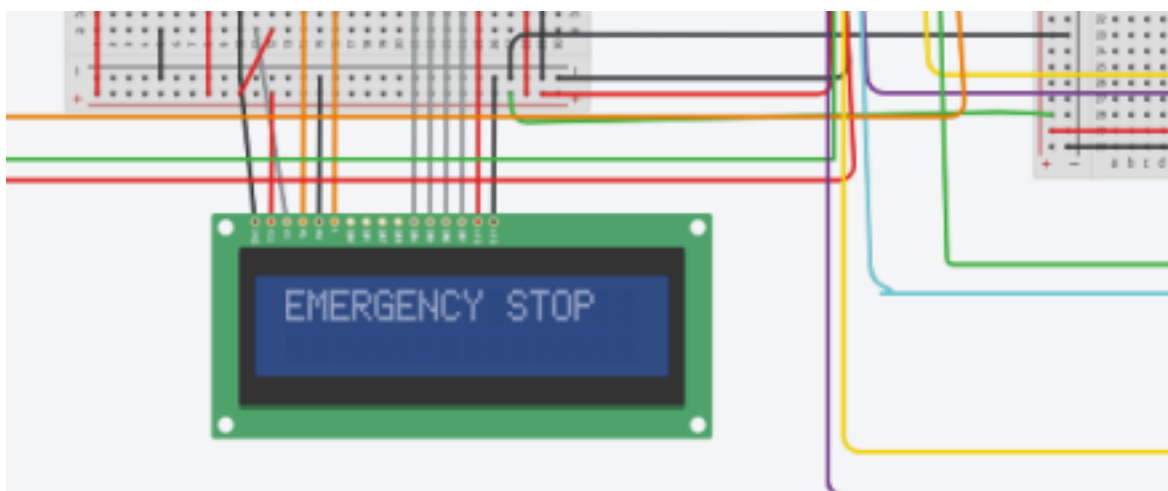
Door Closing After Floor Arrival

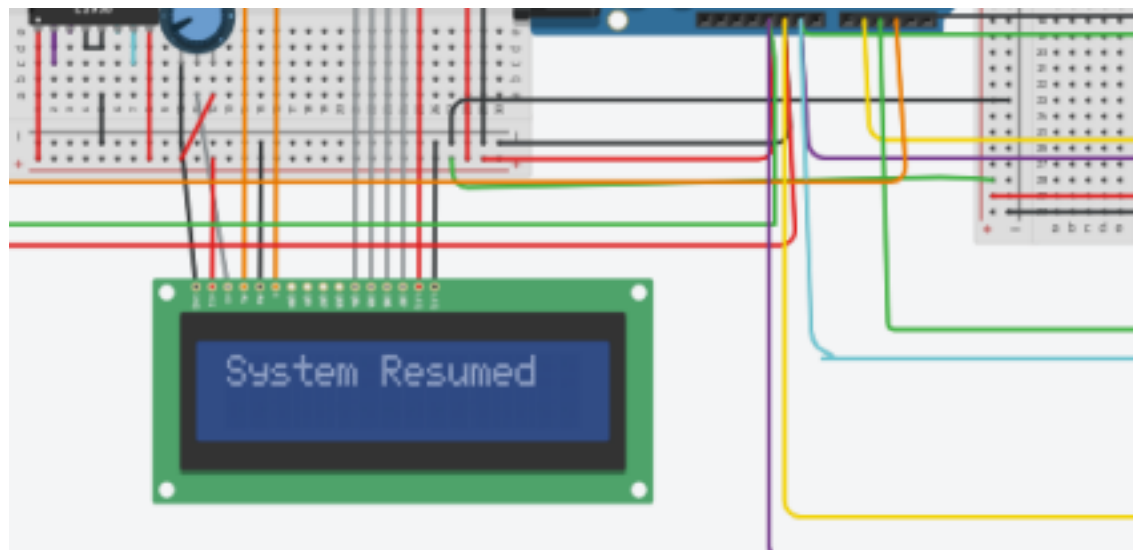


Door Opening Upon Floor Arrival

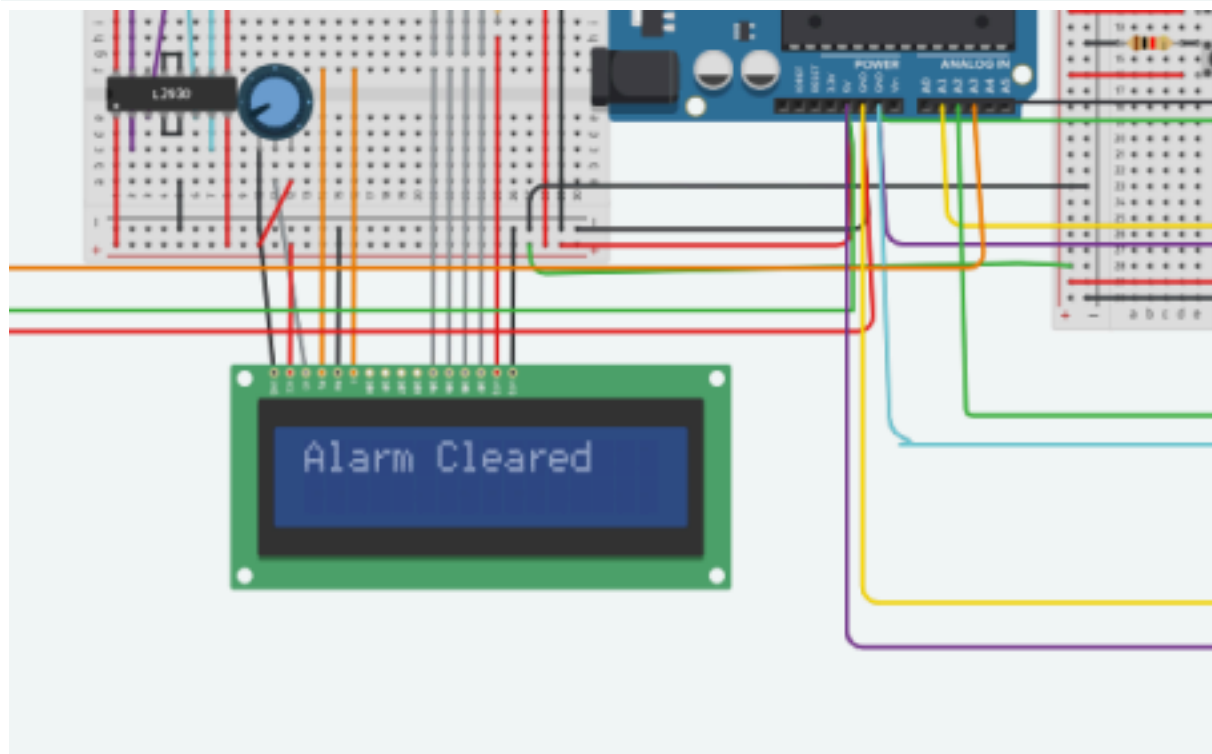
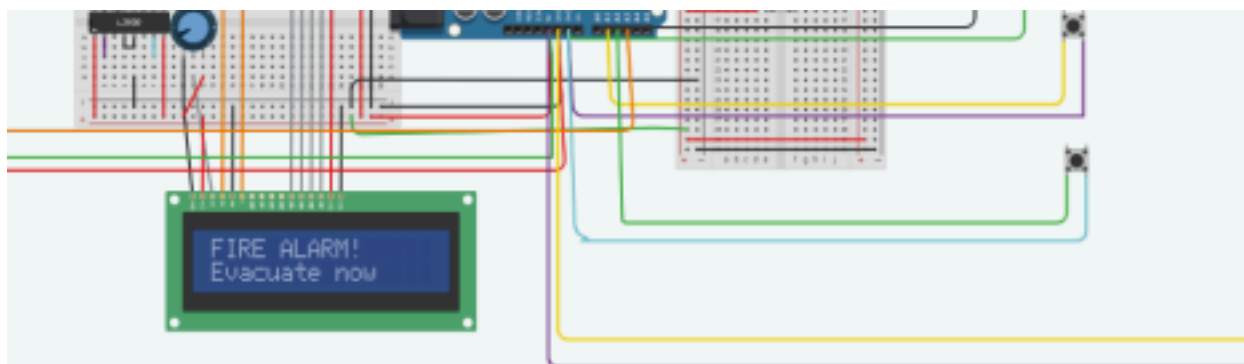


EMERGENCY STOP

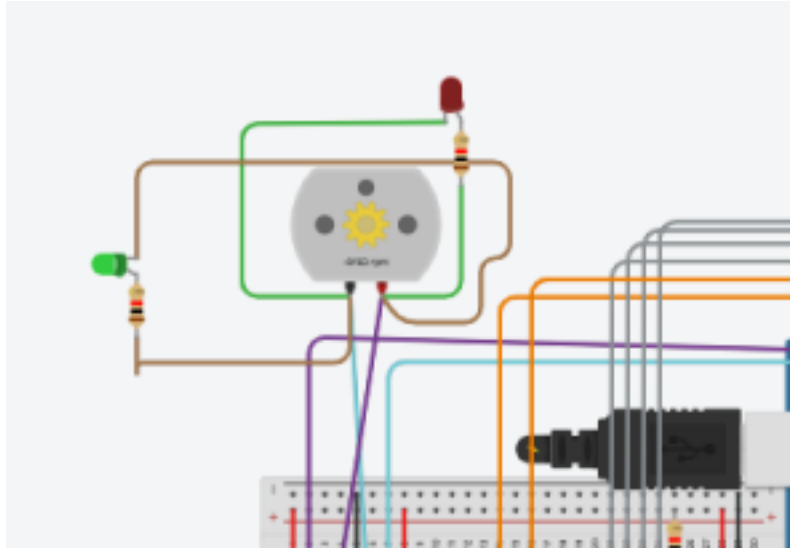




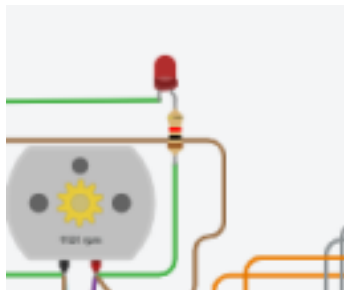
Fire ALARM



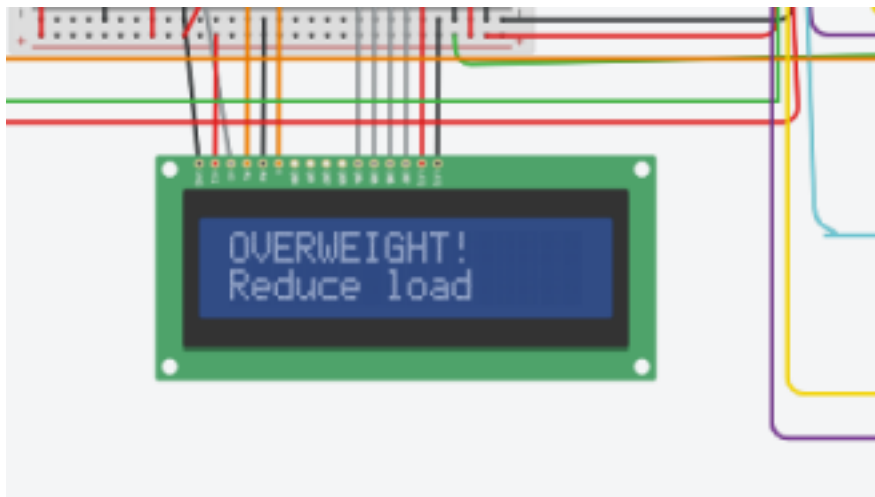
IN IDLE STATE



WHEN ELEVATOR IS BUSY



OVERLOAD:





CODE:

```
#include <LiquidCrystal.h>

#include <Servo.h>


// LCD and Servo

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

Servo doorServo;


// Pins

#define pwm1 9

#define pwm2 10

#define buzzerPin A0

#define btn3 6
#define btn2 7

#define btn1 8

#define fireBtn A1

#define emergencyBtn A2
```



```
#define weightPin A3

#define servoPin 10

#define redLED 29

#define greenLED 30


// Melody

int melody[] = { 262, 294, 330, 349 }; // C, D, E, F notes

int noteDuration = 200;


// States

boolean buttonState1, buttonState2, buttonState3;

bool isEmergency = false, isFireAlarm = false;

bool lastEmergencyBtnState = HIGH, lastFireBtnState = HIGH;

bool welcomeShown = true;

bool isOverweight = false;


int i = 1;

int weightThreshold = 300;
void setup() {

    pinMode(pwm1, OUTPUT);

    pinMode(pwm2, OUTPUT);

    pinMode(buzzerPin, OUTPUT);

    pinMode(redLED, OUTPUT);

    pinMode(greenLED, OUTPUT);
```

```
pinMode(btn1, INPUT_PULLUP);

pinMode(btn2, INPUT_PULLUP);

pinMode(btn3, INPUT_PULLUP);

pinMode(fireBtn, INPUT_PULLUP);

pinMode(emergencyBtn, INPUT_PULLUP);


lcd.begin(16, 2);

Serial.begin(9600);

doorServo.attach(servoPin);


// Welcome message

lcd.setCursor(0, 0); lcd.print("Welcome to Lift");

lcd.setCursor(0, 1); lcd.print("Press floor btn");

// Startup Melody

for (int i = 0; i < 4; i++) {

    tone(buzzerPin, melody[i], noteDuration);

    delay(noteDuration * 1.3);

}

noTone(buzzerPin);


updateLEDStatus(); // show idle state

}
```

```

void loop() {

    updateLEDStatus(); // keep LEDs updated


    buttonState1 = digitalRead(btn1);
    buttonState2 = digitalRead(btn2);
    buttonState3 = digitalRead(btn3);


    if (welcomeShown && (buttonState1 || buttonState2 || buttonState3)) {

        lcd.clear(); welcomeShown = false;

    }


    // Emergency Button
    bool currentEmergencyBtnState = digitalRead(emergencyBtn);

    if (currentEmergencyBtnState == LOW && lastEmergencyBtnState ==
HIGH) {

        isEmergency = !isEmergency;

        lcd.clear();

        lcd.setCursor(0, 0);

        if (isEmergency) {

            lcd.print("EMERGENCY STOP");

            digitalWrite(pwm1, LOW); digitalWrite(pwm2, LOW);

            digitalWrite(buzzerPin, HIGH);

        } else {

            lcd.print("System Resumed");

```

```

    digitalWrite(buzzerPin, LOW);

    delay(1500); lcd.clear();

}

updateLEDStatus();

}

lastEmergencyBtnState = currentEmergencyBtnState;

if (isEmergency) return;


// Fire Alarm

bool currentFireBtnState = digitalRead(fireBtn);

if (currentFireBtnState == LOW && lastFireBtnState == HIGH) {
    isFireAlarm = !isFireAlarm;

    lcd.clear();

    if (isFireAlarm) {

        lcd.print("FIRE ALARM!");

        lcd.setCursor(0, 1); lcd.print("Evacuate now");

        digitalWrite(pwm1, LOW); digitalWrite(pwm2, LOW);

        digitalWrite(buzzerPin, HIGH);

    } else {

        lcd.print("Alarm Cleared");

        digitalWrite(buzzerPin, LOW);

        delay(1500); lcd.clear();

    }

    updateLEDStatus();

```

```

}

lastFireBtnState = currentFireBtnState;

if (isFireAlarm) return;


// Weight Check

int weightValue = analogRead(weightPin);

Serial.print("Weight Reading: "); Serial.println(weightValue);

if (weightValue > weightThreshold) {

    if (!isOverweight) {
        lcd.clear();

        lcd.print("OVERWEIGHT!");

        lcd.setCursor(0, 1); lcd.print("Reduce load");

        digitalWrite(pwm1, LOW); digitalWrite(pwm2, LOW);

        digitalWrite(buzzerPin, HIGH);

        isOverweight = true;
    }

    return;
} else {

    if (isOverweight) {

        lcd.clear();

        lcd.print("Weight Normal");

        digitalWrite(buzzerPin, LOW);

        delay(1500); lcd.clear();

        isOverweight = false;
    }
}

```

```
}  
}
```

```
digitalWrite(pwm1, LOW); digitalWrite(pwm2, LOW);
```

```
// Floor 3
```

```
if (buttonState3 == HIGH && !welcomeShown) {  
  while (i < 3) {  
  
    i++;  
  
    digitalWrite(pwm1, LOW); digitalWrite(pwm2, HIGH);  
  
    lcd.setCursor(0, 0); lcd.print("GOING TO");  
  
    lcd.setCursor(0, 1); lcd.print("floor "); lcd.print(i);  
  
    delay(4000); lcd.clear();  
  
  }  
  
  i = 3;  
  
  floorArrived();  
  
}
```

```
// Floor 2
```

```
if (buttonState2 == HIGH && !welcomeShown) {  
  
  if (i > 2) {  
  
    while (i > 2) {  
  
      i--;  
  
      digitalWrite(pwm1, HIGH); digitalWrite(pwm2, LOW);
```

```

    lcd.setCursor(0, 0); lcd.print("GOING TO");

    lcd.setCursor(0, 1); lcd.print("floor "); lcd.print(i);

    delay(4000); lcd.clear();

}

} else if (i < 2) {
    while (i < 2) {

        i++;

        digitalWrite(pwm1, LOW); digitalWrite(pwm2, HIGH);

        lcd.setCursor(0, 0); lcd.print("GOING TO");

        lcd.setCursor(0, 1); lcd.print("floor "); lcd.print(i);

        delay(4000); lcd.clear();

    }

}

i = 2;

floorArrived();

}

```

// Floor 1

```

if (buttonState1 == HIGH && !welcomeShown) {

    if (i > 1) {

        while (i > 1) {

            i--;

            digitalWrite(pwm1, HIGH); digitalWrite(pwm2, LOW);

            lcd.setCursor(0, 0); lcd.print("GOING TO");

```

```
        lcd.setCursor(0, 1); lcd.print("floor "); lcd.print(i);  
        delay(4000); lcd.clear();  
    }  
}  
  
i = 1;  
  
floorArrived();  
  
}  
}
```

```
void floorArrived() {  
    digitalWrite(pwm1, LOW); digitalWrite(pwm2, LOW);  
  
    lcd.clear(); lcd.print("Door Opening...");  
  
    doorServo.write(0);  
  
    delay(1000);  
  
    lcd.clear(); lcd.print("Reached Floor ");  
  
    lcd.print(i); beep(); delay(1000);  
  
    lcd.clear(); lcd.print("Door Closing...");  
  
    doorServo.write(90); delay(1000); lcd.clear();  
  
}
```

```
void beep() {  
    digitalWrite(buzzerPin, HIGH);  
  
    delay(500);  
  
    digitalWrite(buzzerPin, LOW);  
  
}
```



```

// LED logic based on emergency/fire/button state

void updateLEDStatus() {

    if (isFireAlarm || isEmergency) {

        digitalWrite(greenLED, LOW);

        digitalWrite(redLED, LOW);

        return;

    }

    if (digitalRead(btn1) == HIGH ||

        digitalRead(btn2) == HIGH ||

        digitalRead(btn3) == HIGH) {

        digitalWrite(greenLED, HIGH);

        digitalWrite(redLED, LOW);

    } else {

        digitalWrite(greenLED, LOW);

        digitalWrite(redLED, HIGH);

    }

}

```

features :

1. System Initialization (Displays startup message)
2. Overloading Detection (Monitors weight input)
3. Floor Selection Control (Enables/disables buttons)
4. Floor Selection Buttons (Select floors 1-3)

5. Elevator Movement (Moves to selected floor)
6. Door Operation (Opens and closes doors)
7. Fire Alarm Button (Triggers fire alarm)
8. Emergency Stop Button (Stops elevator in emergency)
9. Buzzer Sound (Provides audio alerts)
10. LED Indicators (Shows system status)
11. Jumper Wires (Connects components together)
12. LCD Display (Shows messages and status)
13. Door Opener/Closer (Servo-controlled door movement)
14. Power Supply (Provides power to system)
15. Motor Driver Control (Controls motor operation)
16. Servo Motor Control (Operates door movements)
17. Emergency LED Indicator (Red LED during emergency)
18. Safety Features (Overload and emergency protection)
19. User Interface (Interactive with buttons)
20. Manual Override (Allows manual control)
21. Auto-Reset after Alarm (Resets system after alert)

Working

1. Starting the System

- a. When you click on start simulation , the **LCD** screen shows "welcome to lift ,press floor button" to indicate the system is active and ready to operate.
- b. The **green LED** lights up, indicating the system is in idle mode,

ready for use.

2. Overloading Function (First Potentiometer)

- a. One **potentiometer** simulates weight input for overloading detection.
- b. When the potentiometer is turned **clockwise**, the LCD shows "Overload" to indicate that the weight has exceeded the limit.
- c. When the potentiometer is turned **counterclockwise**, the LCD displays "Load Reduced" to show that the weight is within the acceptable range.

3. Floor Selection Button Control (Second Potentiometer)

- a. The second **potentiometer** controls whether the floor selection buttons are active.
- b. When the potentiometer is turned **clockwise**, the floor selection buttons are **enabled** and can be used to select a floor.
- c. When the potentiometer is turned **counterclockwise**, the floor selection buttons are **disabled** and cannot be used until the potentiometer is turned back.

4. Floor Selection

- a. There are **3 push buttons** to select floors:
 - i. **Button 1** for the 1st floor
 - ii. **Button 2** for the 2nd floor
 - iii. **Button 3** for the 3rd floor
- b. When a button is pressed, the system checks if the weight is within the acceptable range.
- c. If everything is fine, the system moves the elevator and shows a message on the LCD like "Moving to Floor X" (X being the floor number). The **DC motor** moves the elevator to the selected floor, and the system updates the floor number on the **LCD**.

5. Elevator Movement

- a. The **DC motor**, controlled by the **L293D motor driver**, simulates the elevator's movement up or down.
- b. Once the motor reaches the selected floor, the floor number is displayed on the **LCD**.
- c. The **red LED** lights up, indicating the elevator is moving.

6. Door Operation (Servo Motor)

- a. When the elevator reaches a floor, the **servo motor** opens the doors by rotating 90°. The servo motor makes a sound when the doors open.
- b. After a short time, the servo motor rotates back to close the doors, making another sound.

7. Fire Alarm Button

- a. There is a **push button** for activating a fire alarm.
- b. When pressed, the **LCD** displays "Fire Alarm! Evacuate Now".
- c. Pressing the button again clears the alarm and the **LCD** shows "Alarm Cleared".

8. Emergency Stop Button

- a. A separate **emergency button** is used to stop the elevator in case of emergency.
- b. When pressed, the **LCD** shows "Emergency Stop" and the elevator immediately halts.
- c. Pressing the button again resumes the system, and the **LCD** shows "System Resumed".
- d. The **red LED** also lights up during the emergency situation, indicating that the system has been stopped.

9. **Buzzer Sound**

- a. The **buzzer** provides audio feedback for certain actions:
 - i. It sounds when the **fire alarm button** is pressed, signaling an emergency.
 - ii. It also sounds when the **emergency stop button** is pressed, alerting users to the emergency situation.

10. **LED Indicators**

- a. The **green LED** lights up when the system is in idle mode, indicating the elevator is ready for use.
- b. The **red LED** lights up when the elevator is moving, and also when the emergency stop button is pressed, indicating the elevator has stopped due to an emergency situation.